

Shell 脚本编程教程



Shell 是 Linux/Unix 系统下用户与操作系统内核交互的接口。Shell 脚本 (Shell Script) 是一种用 Shell 命令编写的程序，可以实现自动化任务、批量处理、系统管理等。



1. Shell 介绍

1.1 什么是 Shell

Shell 是一个**命令解释器**，它接收用户输入的命令，并将其交给操作系统内核执行。同时，Shell 也是一种**编程语言**，支持变量、函数、流程控制等，可以编写脚本完成复杂任务。

常见的 Shell 类型：

Shell 类型	说明
sh	Bourne Shell, 最早的 Shell
bash	Bourne Again Shell, Linux 默认 Shell, 功能强大
zsh	Z Shell, 兼容 bash, 扩展丰富
csh / tcsh	C 语言风格的 Shell



本教程以 **bash** 为准, 绝大多数 Linux 系统默认使用 bash。

1.2 查看当前 Shell

bash

```
1 echo $SHELL          # 查看当前使用的 Shell
2 # 输出: /bin/bash
3
4 cat /etc/shells      # 查看系统支持的 Shell 列表
```

1.3 第一个 Shell 脚本

创建一个最简单的脚本文件 **hello.sh** :

bash

```
1 #!/bin/bash
2 echo "Hello, World!"
```

- **#!/bin/bash** 指定脚本的解释器路径。
- **echo** 输出字符串。

2. 快速入门

步骤 1: 创建脚本文件

bash

```
1 | vim first.sh
```

步骤 2：写入内容

bash

```
1 | #!/bin/bash
2 | # 这是我的第一个脚本
3 | echo "当前目录: "
4 | pwd
5 | echo "当前用户: "
6 | whoami
```

步骤 3：添加执行权限

bash

```
1 | chmod +x first.sh
```

步骤 4：执行脚本

bash

```
1 | ./first.sh          # 相对路径执行
2 | # 或
3 | bash first.sh       # 用 bash 解释器执行
4 | # 或
5 | /full/path/first.sh # 绝对路径执行
```

3. Shell 程序：变量

3.1 变量定义与使用

语法：变量名=变量值 （等号两边不能有空格）

bash

```
1 | #!/bin/bash
```

```
2 name="Alice"           # 定义变量
3 age=25
4 echo $name             # 使用变量（推荐 ${name}）
5 echo "年龄: ${age}岁"
```

命名规则：

- 只能包含字母、数字、下划线，不能以数字开头
- 区分大小写
- 习惯上环境变量用大写，普通变量用小写

3.2 只读变量 (readonly)

```
bash
1 #!/bin/bash
2 readonly PI=3.14159
3 PI=3.14           # 报错: readonly variable
```

3.3 删除变量 (unset)

```
bash
1 #!/bin/bash
2 temp="临时数据"
3 unset temp
4 echo $temp        # 输出空
```

3.4 变量类型

类型	说明
局部变量	仅在当前脚本或函数中有效
环境变量	对所有子进程有效，如 <code>PATH</code> 、 <code>HOME</code>
特殊变量	<code>\$0</code> ， <code>\$?</code> ， <code>\$\$</code> 等，见后续章节

3.5 命令替换

将命令的输出赋值给变量：

```
bash
1  #!/bin/bash
2  current_date=$(date +%Y-%m-%d)    # 推荐写法
3  current_time=`date +%H:%M:%S`     # 反引号旧写法
4  echo "今天日期: $current_date"
5  echo "当前时间: $current_time"
```

4. 字符串

4.1 字符串定义

```
bash
1  str1="Hello"          # 双引号（支持变量解析）
2  str2='World'          # 单引号（原样输出）
3  str3=Linux            # 不加引号（不能有空格）
```

4.2 字符串操作

操作	语法	示例
获取长度	<code>\${#字符串}</code>	<code>echo \${#str}</code>
拼接	直接并置	<code>str=\${str1}\${str2}</code>
截取	<code>\${字符串:起始:长度}</code>	<code>\${str:0:5}</code>
替换	<code>\${字符串/旧/新}</code> （第一次） <code>\${字符串//旧/新}</code> （全部）	<code>\${str/o/O}</code>
查找子串位置	<code>expr index "\$str" "字符"</code>	需 <code>expr</code> 命令

示例：

bash

```
1  #!/bin/bash
2  msg="Hello, Linux!"
3  echo "长度: ${#msg}"           # 输出: 13
4  echo "截取前5字符: ${msg:0:5}" # 输出: Hello
5  echo "替换第一个l为L: ${msg/l/L}" # 输出: HeLlo, Linux!
6  echo "替换所有l为L: ${msg//l/L}" # 输出: HeLLo, Linux!
```

5. Shell 程序：参数传递

执行脚本时可以传递参数，例如：`./test.sh arg1 arg2 arg3`

系统变量：

变量	说明
<code>\$0</code>	脚本名称
<code>\$1</code> , <code>\$2</code> , ...	第1个、第2个... 参数（超过9个需 <code>\${10}</code> ）
<code>\$#</code>	参数个数
<code>\$*</code>	所有参数（作为单个字符串）
<code>@</code>	所有参数（每个作为独立字符串）
<code>\$\$</code>	当前进程 ID
<code>\$?</code>	上一条命令的退出状态（0 成功，非0 失败）

示例：`args.sh`

bash

```
1  #!/bin/bash
2  echo "脚本名称: $0"
3  echo "参数个数: $#"
```

```
4  echo "第一个参数: $1"
5  echo "第二个参数: $2"
6  echo "所有参数(\$*): $*"
7  echo "所有参数(\@): @"
```

8 | `echo "当前进程ID: $$"`

执行：

```
bash
1 | $ ./args.sh hello world 123
2 | 脚本名称: ./args.sh
3 | 参数个数: 3
4 | 第一个参数: hello
5 | 第二个参数: world
6 | 所有参数($*): hello world 123
7 | 所有参数($@): hello world 123
8 | 当前进程ID: 12345
```

6. Shell 程序：运算符

6.1 算术运算符

Shell 本身不支持直接算术运算，需要借助 `$(())` 或 `expr`。

运算符	说明	示例
<code>+</code>	加法	<code>\$(a + b)</code>
<code>-</code>	减法	<code>\$(a - b)</code>
<code>*</code>	乘法	<code>\$(a * b)</code>
<code>/</code>	除法	<code>\$(a / b)</code>
<code>%</code>	取余	<code>\$(a % b)</code>
<code>**</code>	幂运算 (bash 支持)	<code>\$(2 ** 3)</code>

示例：

```
bash
1 | #!/bin/bash
2 | a=10
```

```

3  b=3
4  echo "a + b = $((a + b))"      # 13
5  echo "a - b = $((a - b))"      # 7
6  echo "a * b = $((a * b))"      # 30
7  echo "a / b = $((a / b))"      # 3 (整数除法)
8  echo "a % b = $((a % b))"      # 1
9
10 # 使用 expr
11 sum=`expr $a + $b`              # 注意空格
12 echo "sum = $sum"

```

6.2 关系运算符（用于条件判断）

运算符	含义	示例
<code>-eq</code>	等于	<code>[\$a -eq \$b]</code>
<code>-ne</code>	不等于	<code>[\$a -ne \$b]</code>
<code>-gt</code>	大于	<code>[\$a -gt \$b]</code>
<code>-lt</code>	小于	<code>[\$a -lt \$b]</code>
<code>-ge</code>	大于等于	<code>[\$a -ge \$b]</code>
<code>-le</code>	小于等于	<code>[\$a -le \$b]</code>

6.3 布尔运算符

运算符	说明	示例
<code>!</code>	非	<code>[! false]</code>
<code>-o</code>	或 (OR)	<code>[\$a -eq 1 -o \$b -eq 2]</code>
<code>-a</code>	与 (AND)	<code>[\$a -eq 1 -a \$b -eq 2]</code>

6.4 字符串运算符

运算符	说明	示例
<code>=</code>	字符串相等	<code>["\$str1" = "\$str2"]</code>
<code>!=</code>	字符串不等	<code>["\$str1" != "\$str2"]</code>
<code>-z</code>	长度为零	<code>[-z "\$str"]</code>
<code>-n</code>	长度非零	<code>[-n "\$str"]</code>

6.5 文件测试运算符

运算符	说明	示例
<code>-e</code>	文件存在	<code>[-e /etc/passwd]</code>
<code>-f</code>	普通文件	<code>[-f /etc/passwd]</code>
<code>-d</code>	目录	<code>[-d /home]</code>
<code>-r</code>	可读	<code>[-r file.txt]</code>
<code>-w</code>	可写	<code>[-w file.txt]</code>
<code>-x</code>	可执行	<code>[-x script.sh]</code>
<code>-s</code>	非空文件	<code>[-s file.txt]</code>

7. 流程控制

7.1 if 语句

单分支：

```
bash
1  if [ 条件 ]; then
2      命令
3  fi
```

双分支：

```
bash
1  if [ 条件 ]; then
2      命令1
3  else
4      命令2
5  fi
```

多分支：

```
bash
1  if [ 条件1 ]; then
2      命令1
3  elif [ 条件2 ]; then
4      命令2
5  else
6      命令3
7  fi
```

示例：

```
bash
1  #!/bin/bash
2  read -p "请输入分数：" score
3  if [ $score -ge 90 ]; then
4      echo "优秀"
5  elif [ $score -ge 60 ]; then
6      echo "及格"
7  else
8      echo "不及格"
9  fi
```

7.2 case 语句

```
bash
1  case 变量 in
```

```
2      模式1)
3          命令1
4          ;;
5      模式2)
6          命令2
7          ;;
8      *)
9          默认命令
10         ;;
11     esac
```

示例：

```
bash
1  #!/bin/bash
2  read -p "输入 yes/no: " answer
3  case $answer in
4      yes|y|Y)
5          echo "您选择了是"
6          ;;
7      no|n|N)
8          echo "您选择了否"
9          ;;
10     *)
11         echo "输入无效"
12         ;;
13     esac
```

7.3 for 循环

语法 1：遍历列表

```
bash
1  for 变量 in 列表; do
2      命令
3  done
```

示例：

```
bash
1  #!/bin/bash
2  for i in 1 2 3 4 5; do
3      echo "Number: $i"
4  done
5
6  for name in Alice Bob Charlie; do
7      echo "Hello, $name"
8  done
9
10 # 遍历当前目录所有 .txt 文件
11 for file in *.txt; do
12     echo "处理 $file"
13 done
```

语法 2：C 语言风格

```
bash
1  for ((初始值; 条件; 步进)); do
2      命令
3  done
```

示例：

```
bash
1  #!/bin/bash
2  for ((i=0; i<10; i++)); do
3      echo "i = $i"
4  done
```

7.4 while 循环

```
bash
1  while [ 条件 ]; do
2      命令
```

示例：

```
bash
1  #!/bin/bash
2  count=1
3  while [ $count -le 5 ]; do
4      echo "count = $count"
5      ((count++))
6  done
```

7.5 循环控制

命令	说明
<code>break</code>	跳出当前循环
<code>continue</code>	跳过本次循环剩余部分，进入下一次迭代

示例：

```
bash
1  #!/bin/bash
2  for i in {1..10}; do
3      if [ $i -eq 5 ]; then
4          break
5      fi
6      echo $i
7  done
8  # 输出 1 2 3 4
9
10 for i in {1..5}; do
11     if [ $i -eq 3 ]; then
12         continue
13     fi
14     echo $i
```



```
15 | done
16 | # 输出 1 2 4 5
```

8. 函数使用

8.1 定义函数

语法 1:

```
bash
1 | function 函数名 {
2 |     命令
3 | }
```

语法 2:

```
bash
1 | 函数名() {
2 |     命令
3 | }
```

8.2 函数参数

函数内部可以通过 `$1`、`$2` ... 获取参数，`$#` 获取参数个数。

示例:

```
bash
1 | #!/bin/bash
2 | # 定义函数
3 | say_hello() {
4 |     echo "Hello, $1!"
5 | }
6 |
7 | add() {
```

```

8      local sum=$(( $1 + $2 ))    # local 定义局部变量
9      echo "和: $sum"
10  }
11
12  # 调用函数
13  say_hello "Alice"
14  add 10 20

```

8.3 函数返回值

- **return**: 返回整数 (0~255)，通过 `$?` 获取
- **echo**: 返回字符串，通过命令替换获取

```

1  #!/bin/bash
2  # 方法1: 使用 return
3  check_file() {
4      if [ -f "$1" ]; then
5          return 0    # 成功
6      else
7          return 1    # 失败
8      fi
9  }
10 check_file "/etc/passwd"
11 if [ $? -eq 0 ]; then
12     echo "文件存在"
13 fi
14
15 # 方法2: 使用 echo 返回字符串
16 get_date() {
17     echo $(date +%Y-%m-%d)
18 }
19 today=$(get_date)
20 echo "今天日期: $today"

```

8.4 删除函数

```
bash
1 unset -f 函数名
```

9. 数组

9.1 定义数组

```
bash
1 # 方式1: 逐个赋值
2 arr[0]="apple"
3 arr[1]="banana"
4
5 # 方式2: 括号赋值
6 fruits=("apple" "banana" "cherry")
7
8 # 方式3: 指定索引
9 colors=[0]="red" [2]="blue"
```

9.2 访问数组元素

```
bash
1 echo ${fruits[0]}      # apple
2 echo ${fruits[1]}      # banana
3 echo ${fruits[@]}      # 所有元素
4 echo ${fruits[*]}      # 所有元素（作为字符串）
5 echo ${#fruits[@]}     # 数组长度
6 echo ${!fruits[@]}     # 获取所有索引（键）
```

9.3 遍历数组

```
bash
1  #!/bin/bash
2  fruits=("apple" "banana" "cherry")
3
4  # 方法1: 通过索引
5  for i in "${!fruits[@]}"; do
6      echo "$i: ${fruits[$i]}"
7  done
8
9  # 方法2: 直接遍历值
10 for fruit in "${fruits[@]}"; do
11     echo "水果: $fruit"
12 done
```

9.4 添加/删除元素

```
bash
1  # 添加元素
2  fruits+=("date")
3  fruits[5]="elderberry"
4
5  # 删除元素（使用 unset）
6  unset fruits[1]      # 删除索引1的元素
7  unset fruits         # 删除整个数组
```

10. 加载其它文件的变量

通过 `source` 命令或 `.` 可以在当前 Shell 环境中执行脚本，使其定义的变量、函数在当前脚本中可用。

语法：

bash

```
1 source 文件名
2 . 文件名 # 点命令，等效于 source
```

示例：创建 `config.sh`

bash

```
1 #!/bin/bash
2 # config.sh
3 USERNAME="admin"
4 PASSWORD="123456"
5 DB_NAME="mydb"
```

在主脚本中加载：

bash

```
1 #!/bin/bash
2 # main.sh
3 source ./config.sh
4 echo "用户名: $USERNAME"
5 echo "密码: $PASSWORD"
```

11. Shell 脚本综合案例：猜数字游戏

11.1 游戏规则

1. 程序随机生成一个 1~100 之间的整数。
2. 用户在终端中输入猜测的数字。
3. 程序根据用户的输入给出提示：
 - 如果猜对了，显示“恭喜猜中！”并结束游戏。
 - 如果猜大了，显示“猜大了”。
 - 如果猜小了，显示“猜小了”。

4. 用户可以随时输入 `q` 或 `quit` 退出游戏。

5. 游戏会记录猜测次数，猜中后显示总次数。



Bash 提供了一个内置变量 `RANDOM`，每次引用它都会返回一个介于 `0~32767` 之间的随机整数。

11.2 程序运行效果图

以下是在终端中运行脚本的完整交互过程：

```
text
1  $ ./guess.sh
2  =====
3      欢迎来到猜数字游戏
4  =====
5  系统已随机生成一个 1~100 之间的整数
6  输入 q 可退出游戏
7
8  请输入你猜的数字： 50
9  猜小了
10
11  请输入你猜的数字： 75
12  猜大了
13
14  请输入你猜的数字： 62
15  猜小了
16
17  请输入你猜的数字： 68
18  恭喜！猜中了！答案是 68
19  你一共猜了 4 次。
20  =====
```

从效果图中可以看到，脚本需要读取用户在键盘上输入的内容，并根据不同的输入执行不同的逻辑。这需要用到 Shell 编程中非常重要的命令 —— `read`。

11.3 read 命令详解

read 命令用于从标准输入（通常是键盘）读取用户输入，并将其存储到一个或多个变量中。

语法结构： `read [选项] [变量名...]`

常用选项

选项	说明
<code>-p "提示信息"</code>	在等待输入前输出提示文字（最常用）
<code>-t 秒数</code>	设置超时时间，超过后自动退出读取
<code>-n 字符数</code>	只读取指定数量的字符（无需按回车）
<code>-s</code>	静默模式，输入不显示（适合密码）
<code>-r</code>	原样读取，不解释反斜杠转义

基本用法示例

		bash
1	# 最简单的读取	
2	<code>read name</code>	
3	<code>echo "你输入的是: \$name"</code>	
4		
5	# 带提示信息（推荐）	
6	<code>read -p "请输入你的年龄: " age</code>	
7	<code>echo "年龄: \$age"</code>	
8		
9	# 读取多个变量（用空格分隔输入）	
10	<code>read -p "请输入姓名 城市: " name city</code>	
11	<code>echo "姓名: \$name, 城市: \$city"</code>	
12		
13	# 静默读取（不显示输入内容）	
14	<code>read -s -p "请输入密码: " password</code>	


```

15  echo    # 换行
16  echo "密码已录入"
17
18  # 超时读取
19  read -t 5 -p "5秒内输入: " quick
20  if [ -z "$quick" ]; then
21      echo "超时未输入"
22  fi

```



注意：如果不指定变量名，输入的内容会保存在系统变量 `$REPLY` 中。

11.4 猜数字游戏完整脚本

```

bash
1  #!/bin/bash
2  # guess.sh - 猜数字游戏
3
4  # 生成随机数 (1-100)
5  target=$((RANDOM % 100 + 1))
6  count=0
7
8  # 显示欢迎信息
9  echo "===== "
10 echo "      欢迎来到猜数字游戏      "
11 echo "===== "
12 echo "系统已随机生成一个 1~100 之间的整数"
13 echo "输入 q 可退出游戏"
14 echo
15
16 # 游戏主循环
17 while true; do
18     read -p "请输入你猜的数字: " guess
19

```

```
20      # 检测退出指令
21      if [ "$guess" = "q" ] || [ "$guess" = "quit" ]; then
22          echo "游戏已退出，正确答案是 $target"
23          exit 0
24      fi
25
26      # 检查输入是否为有效数字
27      if ! [[ "$guess" =~ ^[0-9]+$ ]]; then
28          echo "输入无效，请输入一个整数！"
29          continue
30      fi
31
32      # 计数并判断
33      ((count++))
34      if [ $guess -eq $target ]; then
35          echo "恭喜！猜中了！答案是 $target"
36          echo "你一共猜了 $count 次。"
37          echo "===== "
38          exit 0
39      elif [ $guess -gt $target ]; then
40          echo "猜大了"
41      else
42          echo "猜小了"
43      fi
44  done
```

11.5 脚本知识点解析

知识点	在本脚本中的应用
<code>\$((RANDOM % 100 + 1))</code>	生成 1~100 随机数
<code>read -p "提示" guess</code>	读取用户输入并存入 <code>guess</code> 变量
<code>if ["\$guess" = "q"]</code>	判断用户是否输入退出指令
<code>[["\$guess" =~ ^[0-9]+\$]]</code>	正则匹配，确保输入为纯数字
<code>((count++))</code>	算术运算，记录猜测次数
<code>if ... elif ... else</code>	比较大小并给出相应提示
<code>exit 0</code>	正常退出脚本

11.6 扩展练习

- 限制猜测次数**：添加最多 10 次的机会，超时后提示失败并退出。
- 难度选择**：让用户选择数字范围（如 1-50、1-200）。
- 记录最佳成绩**：将最少猜测次数保存到文件，下次运行时显示历史记录。